M06 Task Management System

Methodology Intermediate

Organize Claude's work with the Tasks API

v2.1.233: on Opus 4.8, Sonnet 5, Fable 5, Mythos 5, and newer models, the Tasks API (and TodoWrite) are disabled by default. Re-enable with `CLAUDE_CODE_ENABLE_TODO_TOOLS=1`.

Task hierarchy

When enabled, the Tasks API (v2.1.16+) supports persistent status and dependency tracking. The following four levels are a planning convention, not required task types.

Level	Role
Plan	High-level objective
Phase	Major milestone
Task	Concrete deliverable
Subtask	Atomic action

Tasks survive session endings, context compactions, and restarts. They are stored in `~/.claude/tasks/<id>/`.

Core tools

```
TaskCreate   Create a task or subtask
TaskList     List all tasks (statuses + blockages)
TaskGet      Read full details of a task
TaskUpdate   Update status or metadata
TaskStop     Stop a running background execution
```

Execution cycle

```
# 1. Opt in on newer models and use a stable list ID
export CLAUDE_CODE_ENABLE_TODO_TOOLS=1
export CLAUDE_CODE_TASK_LIST_ID="myproject-auth-v2"
claude

# 2. List and resume
> "TaskList to see current state"

# 3. Mark a task in progress then complete it
> "Mark task-login as in_progress, then implement"
> "Mark task-login as completed"
```

Available statuses

Status	Meaning
`pending`	Waiting
`in_progress`	Currently executing
`completed`	Successfully finished

Dependencies with blockedBy

The `blockedBy` field prevents premature execution of a task that depends on another. The IDs used must be the actual task identifiers, not their titles.

```
Create "Write integration tests", then set its dependencies
using the actual IDs returned for the login and refresh tasks.
```

Pending tasks with unresolved dependencies cannot be claimed. Record failures in the description or metadata; the task states are `pending`, `in_progress`, and `completed`.

When to activate the Tasks API

Use it when dependencies or state must survive a session. A small task may need only a short plan; both task systems require the opt-in on newer models.

Cost to anticipate

`TaskList` returns a summary; `TaskGet` retrieves details for a selected task. These are tool calls, not a one-to-one count of billed model requests. Put useful summaries in `subject` and retrieve detailed descriptions when needed to limit unnecessary context.

List ID: naming rules

A generic ID shared between projects causes collisions. Use a repo-specific identifier in the form `<org>-<repo>-<feature>`.